

OMES-DEV-001 · Revision D

OpenCore MES

Developer's guide

Installation, services and integration

Document	OMES-DEV-001
Revision	D, issued 2026-10-05
Applies to	OpenCore MES, the platform as built on the issue date

Contents

1. 1. About this guide

- 2. 1.1 Who it is for
- 3. 1.2 What is built, and what is planned
- 4. 1.3 Conventions

5. 2. Installing OpenCore MES

- 6. 2.1 What you need
- 7. 2.2 Settings
- 8. 2.3 Sign-in
- 9. 2.4 Scaling out

10. 3. How integration works

- 11. 3.1 The two elements
- 12. 3.2 Data flows
- 13. 3.3 Identity: who a service acts as
- 14. 3.4 Governance: deny by default, approved by whoever answers

15. 4. A first integration, step by step

16. 5. Reference

- 17. 5.1 Connection
- 18. 5.2 Service
- 19. 5.3 The script contract
- 20. 5.4 Refusals, faults and retries
- 21. 5.5 Triggers: exactly once to enqueue, at least once to run
- 22. 5.6 Schedules
- 23. 5.7 The web service API
- 24. 5.8 Limits
- 25. 5.9 Security
- 26. 5.10 Testing
- 27. 5.11 Running it: nodes, settings, the monitor

28. 6. Recipes

- 29. 6.1 Inbound without duplicates
- 30. 6.2 Saying no with the right field
- 31. 6.3 Down is not no
- 32. 6.4 Choosing the identity
- 33. 6.5 Keeping a run inside the right network

34. 7. Other ways in

- 35. 7.1 The AI design API
- 36. 7.2 Versions and notice
- 37. 7.3 Excel import and export
- 38. 7.4 What is not an interface
- 39. 7.5 Planned

40. 8. Appendix

- 41. 8.1 Review checklist for a service change
- 42. 8.2 Where it is in the code
- 43. 8.3 Revision history

Procedures

- 1. IN-01 Install for development
- 2. IN-02 Install on a server
- 3. IN-03 Upgrade to a new release
- 4. DV-01 Give ERP an identity and a token
- 5. DV-02 Describe ERP as a connection
- 6. DV-03 Write the web service ERP calls
- 7. DV-04 Write the trigger that confirms a released lot
- 8. DV-05 Dry-run the scripts and keep test cases
- 9. DV-06 Submit, approve, and set the secret
- 10. DV-07 Call it from ERP, and watch it

1. About this guide

1.1 Who it is for

For IT, who installs OpenCore MES; for developers who connect OpenCore MES to other systems (ERP, a LIMS, a label printer, a data platform), for the designers who write service scripts, and for IT, who issues tokens, sets secrets and runs the nodes. It describes what OpenCore MES offers today, with the limits the code holds, and marks what is **planned**.

The ground rule: **integration is designed, not deployed**. A web service, a trigger, a schedule and the outside systems they reach are design elements, drafted in a change request, reviewed, approved by the stewards of everything they reach, and live the moment the change executes: no build, no deploy, no restart. Nothing reaches the plant around that lifecycle, and every record a service reads or writes goes through the same policies, rule pipe and audit trail as a person at a form.

1.2 What is built, and what is planned

Way in or out	What	Status
Web services	Outside systems call <code>POST /svc/v1/<service></code> with a bearer token; <code>GET /svc/v1/openapi.json</code> describes what the token's user may call	built
Connections	Outside systems services call, with only the requests their design allows, credentials kept as server secrets	built
Record triggers	A record event (create, update, archive, restore, or a transition) sets a service off after its commit, retried while the other system is down	built
Schedules	The clock sets a service off: every N minutes or hours, at times of day, in a window, on days, or at the times a kind of schedule an installed suite adds works out; catch-up and overlap rules	built
The integration monitor	Nodes, schedules, inbound and outbound calls, triggers; pause, resume, run now	built
The AI design API	<code>/ai/v1</code> : an AI drafts and tests designs as a person, never approves	built
Excel import and export	Records in and out as spreadsheets, through the forms' checks	built
Authentication	Single sign-on (OpenID Connect), the plant's directory (LDAP, Active Directory), or a password of OpenCore MES's own; lockout; only people in People & departments (2.3)	built
Versioned HTTP APIs	<code>/svc/v1</code> and <code>/ai/v1</code> written down as a contract (<code>docs/contracts/http-apis</code>): <code>API-Version</code> on every answer, notice before a change (Deprecation, Sunset), the API kit; a plant's web services held to notice before they break their callers (7.2)	built
The equipment adapter contract	What an adapter for one machine protocol implements (<code>docs/contracts/equipment-adapter</code>): its specification, schema and conformance kit (§31.4)	built
The extension API	A narrow, versioned surface for suites, with its conformance kit (§31.3)	planned
Equipment data collection	High-rate machine data handled outside the MES, which receives only derived events (§15.1)	planned

1.3 Conventions

- `Code` is typed as it stands. JSON designs are shown as the designer's **JSON** tab stores them.
- §n refers to a section of the design document, kept by the maintainers.
- The examples use the seed's **work order** and **lot**, and a made-up ERP at `https://erp.plant.local/api`. The same designs are proved end to end by `app/mes/test/integration.mjs`, against a fake ERP.
- Procedures (DV-...) have a **Done** box per step, for a printed copy.

2. Installing OpenCore MES

OpenCore MES is plain JavaScript on Node and PostgreSQL: no build step, two runtime packages (`pg` , and the Anthropic SDK for the copilot). One Node process is one instance; instances share one database.

Caution. In development (and on a public demo, `DEM0=1` , reset every night) sign-in is a **picker**: a person picks who they are, so anyone who can reach that instance can act as anyone. Keep development on `HOST=127.0.0.1` or a network you trust. With `PROD=1` the picker is off and people sign in as 2.3 describes.

2.1 What you need

What	Version	Why
Node	24 or later	the server, the script runner, the tools
PostgreSQL	16 or later (the pipeline runs on 16)	everything that is kept: designs, records, audit, outbox, sessions
Google Chrome or Chromium	any recent	only to build the printable guides (<code>--pdf</code>)

IN-01 Install for development

Purpose	A working instance on your machine, with the seed's plant to try things on.
Done by	Developer
Before you start	Node 24 and PostgreSQL are installed; your user can create databases over the local socket.

#	Do this	You should see	Done
1	Get the code and run <code>npm install</code> .	Two packages installed.	
2	<code>npm run db:reset</code>	The database <code>openmes_poc</code> is made (when missing), its schema loaded and migrated, and the seed loaded: people, departments, objects, records. It says what it loaded.	
3	Optionally, copy <code>.env.example</code> to <code>.env</code> and set what you need (the copilot's provider, a port).	<code>.env</code> is never committed.	
4	<code>npm run dev</code>	"listening on http://127.0.0.1:9090". It restarts when a file under <code>app/mes</code> or <code>src</code> changes, and every open page reloads.	
5	Open it and pick a person to sign in as (Dana designs; Olga works the floor).	The navigator, the designer, the screens.	
6	<code>npm run test:all</code>	Every stage passes. It resets <code>openmes_test</code> (<code>TEST_DATABASE_URL</code>) and refuses any database whose name lacks "test", so your development database is safe.	

`DATABASE_URL` names another database; `npm run db:reset -- --blank` makes one with people only (every model designed from nothing), as the training instance does (`npm run training:reset`, `npm run training`, port 9091).

IN-02 Install on a server

Purpose	One instance behind HTTPS, run by systemd, as the public demo is (ops/demo).
Done by	IT
Before you start	A fresh Ubuntu 24.04 server, root access, a DNS name pointing at it.

#	Do this	You should see	Done
1	As root, run <code>ops/demo/provision.sh</code> (safe to run again).	PostgreSQL, Caddy, Node (its checksum checked); the system user <code>openmes</code> ; its database role over the local socket (peer authentication, no password); the role <code>mes_query</code> the query page reads as, granted to <code>openmes</code> .	
2	Unpack a release in <code>/srv/open-mes/releases/<build></code> , owned by <code>openmes</code> , and run <code>npm ci --omit=dev</code> there.	—	
3	Write its settings in an environment file (as <code>ops/demo/demo.env</code>): <code>PROD=1</code> , <code>HOST=127.0.0.1</code> , <code>PORT=3000</code> , <code>DATABASE_URL=postgres:///openmes</code> , <code>PGHOST=/var/run/postgresql</code> , <code>PLANT_TZ</code> , <code>EVENT_LOG_DIR=/var/lib/open-mes/events</code> . Secrets (the copilot's key, <code>MES_SECRET_...</code>) go in a file of their own under <code>/etc/open-mes</code> , mode 0640, root:openmes.	No secret is in the release or its settings file.	
4	The first time only, as <code>openmes</code> with those settings: <code>node app/mes/db/reset.mjs</code> (add <code>--blank</code> for people only).	The database is made and seeded. Later releases migrate it at start.	
5	Point <code>/srv/open-mes/current</code> at the release; install the systemd unit (as <code>ops/demo/open-mes.service</code> : <code>ExecStart=/usr/local/bin/node app/mes/server.mjs</code> , the environment files, <code>ProtectSystem=strict</code> , <code>ReadWritePaths=/var/lib/open-mes</code>); <code>systemctl enable --now open-mes</code> .	<code>systemctl status open-mes</code> shows it running.	
6	Put Caddy in front (as <code>ops/demo/Caddyfile</code> : your name, <code>reverse_proxy 127.0.0.1:3000</code> , <code>HSTS</code>), and reload it.	Caddy fetches the certificate by itself.	
7	<code>curl -fsS http://127.0.0.1:3000/healthz</code> , then open <code>https://<name>/</code> .	The health answer; the sign-in page over HTTPS.	

`ops/demo/deploy.sh` does all of this from a workstation for the public demo (`--provision` the first time), runs the test pipeline on the server before switching, and keeps the last five releases. It is the demo's: it also installs the timer that erases the demo's database every night, and the landing page. Take the steps from it for a plant, not the script as it is.

IN-03 Upgrade to a new release

Purpose	A new version of OpenCore MES, with its database changes.
Done by	IT

#	Do this	You should see	Done
1	Unpack the release beside the current one and run <code>npm ci --omit=dev</code> .	—	
2	Optionally run the pipeline there against a test database: <code>TEST_DATABASE_URL=postgres:///openmes_test npm run test:all</code> .	Every stage passes.	
3	Point <code>current</code> at it and restart the service.	At start the server runs every new or changed migration, in order, under one lock: several instances starting together run each once. Nothing is run by hand.	
4	<code>/healthz</code> , and the event log.	<code>db.migrated</code> names what ran.	

Designs are never part of a release: they live in the database and change only through change requests, so an upgrade leaves the plant's designs as they were. `npm run db:migrate` migrates without starting a server.

2.2 Settings

Setting	What
<code>DATABASE_URL</code>	the database (default <code>postgres:///openmes_poc</code> , the local socket)
<code>PORT</code> , <code>HOST</code>	where it listens (9090; every IPv4 address, <code>::</code> for IPv6 too, <code>127.0.0.1</code> for this machine only)
<code>PROD=1</code> , <code>BUILD</code>	production: a snapshot of the pages, minified, Secure cookies; this release's name
<code>DEMO=1</code> , <code>DEMO_AS</code>	a public demo: no sign-in page, a visitor is the demo person (<code>DEMO_AS</code> , or the first who designs) and becomes anyone from the top bar; every page says so; services call no outside system
<code>REPLICA_URL</code>	a streaming replica for reads, behind the replay fence (2.4)
<code>BUS=1</code> , <code>INSTANCE</code>	join the change bus, to run several instances on one database; this instance's name
<code>SESSION_COOKIE</code>	the sign-in cookie's name, for a second instance on the same machine
<code>SIMPLE_LISTS=1</code>	lists of people show everyone at once, as in development and on the demo (a test instance); otherwise they start with a search box
<code>OIDC_ISSUER</code> , <code>LDAP_URL</code> , <code>PASSWORDS=0</code> , <code>PICKER=1</code>	how people sign in (2.3)
<code>AI_PROVIDER</code> ...	the copilot: <code>anthropic</code> (Claude: <code>ANTHROPIC_API_KEY</code> , <code>AI_MODEL</code> , <code>AI_EFFORT</code>) or <code>openai</code> (any OpenAI-compatible model: <code>AI_BASE_URL</code> , <code>AI_API_KEY</code> , <code>AI_MODEL</code>); unset, no copilot
<code>MES_SECRET_<NAME></code>	a connection's secret (5.9)

Setting	What
<code>SCRIPT_RUNNER_WRAP</code> , <code>SCRIPT_ISOLATION=required</code>	the script runner's operating-system walls (<code>ops/script-runner-sandbox.sh</code> , Linux), and refusing to start without them (5.9)
<code>OUTBOX=0</code> , <code>SCHEDULER=0</code> , <code>NODE_TAGS</code> , <code>PLANT_TZ</code>	what this node runs for integration (5.11)
<code>EVENT_LOG_DIR</code>	where the event log is written (default <code>.local/events</code>)
<code>SEED_BLANK=1</code>	for <code>db/reset.mjs</code> : people only

2.3 Sign-in

Who may sign in is People & departments' to say: a person active there, nobody else. Someone the plant's identity provider or directory knows, but People & departments does not, is refused by name and told to ask whoever keeps it. How people prove who they are is any of these, together:

Way	Settings	Notes
Single sign-on (OpenID Connect)	<code>OIDC_ISSUER</code> , <code>OIDC_CLIENT_ID</code> , <code>OIDC_CLIENT_SECRET</code>	Register <code>https://<this site>/login/sso/callback</code> with the provider, or set <code>OIDC_REDIRECT_URI</code> . The authorization code flow with PKCE; the ID token's signature, issuer, audience, expiry and nonce are checked. The sign-in id is the <code>preferred_username</code> claim, or <code>OIDC_CLAIM</code> . <code>OIDC_LABEL</code> names the button. Multi-factor is the provider's.
The plant's directory (LDAP, Active Directory)	<code>LDAP_URL</code> , <code>LDAP_USER_DN</code>	A bind as the person with the password they typed: <code>ldaps://dc.plant:636</code> , and <code>{user}@plant.local</code> (AD) or <code>uid={user},ou=people,dc=plant,dc=example</code> . <code>LDAP_CA_FILE</code> for a plant CA. <code>ldap://</code> sends passwords in the clear: the server warns.
A password of OpenCore MES's own	on unless <code>PASSWORDS=0</code>	For plants without either. IT gives a one-time link, valid 72 hours: <code>node app/mes/db/password.mjs olga --url https://mes.plant</code> . The person sets it there (12 characters at least) and changes it from their name at the top right; a change signs them out elsewhere.
The picker	development, <code>DEMO=1</code> , or <code>PICKER=1</code>	Anyone as anyone, no password. Never on an instance others can reach.

A person with a password here signs in with it, anyone else through the directory. Five wrong passwords in a row lock the sign-in id for 15 minutes. Sign-ins (and the way), refusals (and why), sign-outs, links and passwords set are in the audit trail under `$auth`. At start, the server prints the ways that are on. Planned: password ageing, SAML.

2.4 Scaling out

- **A read replica.** Writes go to the primary; reads a replica can serve go to it once it has replayed every write this process committed (the replay fence), waiting at most 250 ms before the primary answers instead. Sessions, idempotency keys, the write path and the designer stay on the primary.
- **Several instances.** `BUS=1` on each: every change's targets reach every instance, so a save on one re-runs the open lists on all. Put a balancer in front that keeps a browser on one instance (its live stream and its calls must meet): `app/mes/lb.mjs` for a trial (`LB_PORT`, `BACKENDS`),

HAProxy, nginx or a cloud balancer in production, with the same rule. `/healthz` answers 503 while an instance drains.

- **Nodes for integration.** Every instance is a node: it may plan schedules and run triggers. Tag the ones inside the network that reaches ERP (`NODE_TAGS=erp`) and keep those services to them (5.11).

`app/mes/README.md` has the commands and the measurements (a replica took 8.6 times fewer commits off the primary; three instances shared 100 saves a second evenly).

3. How integration works

3.1 The two elements

Element	What it is	Holds
Connection	An outside system	its base URL; its authentication (<code>none</code> , <code>bearer</code> , <code>header</code> , <code>basic</code>) naming a secret ; allow , the only requests services may send it; its timeout; its stewards
Service	A script, and what may set it off	its input; whether it is a web service; its callers; its triggers (record events, schedules); who it acts as; what it may reach (objects and connections); the node tag its runs keep to; its stewards

A service is set off four ways, all running the same script the same way: an outside system calls it over HTTP; a person calls it from the designer (**Try it**); a record event; the clock. Whichever, it runs as the identity its design names, and reaches only what its design declares.

3.2 Data flows

Inbound ERP → POST /svc/v1/erp_work_order_in (token: user erp) → callers checked → input checked → script → `ctx.records.create("work_order", ...)` as the service's identity → work order's policy, rule pipe, audit → `{ output }`

Outbound a lot released (any form, transaction, service or import) → in the same database transaction, an outbox row → after the commit, a node's outbox worker → script → `ctx.http("erp", ...)` : allowed request, secret added on the server → ERP → done, or retried while ERP is down

Scheduled the clock → the scheduler plans the run once, whatever the number of nodes → outbox → script with `ctx.event.previous.output` (its cursor) → its next page at once if it asks for more

3.3 Identity: who a service acts as

Its design's **Identity** tab (`runAs`) says who it acts as. Whoever it is, every record it touches is checked as that identity, exactly as at a form, and the audit names on whose behalf it acted.

runAs	Acts as	Use it when
<code>service</code> (the default)	its own identity, <code>service:<name></code> , holding exactly the object roles its design grants (<code>"roles": { "work_order": ["planner"] }</code>), on behalf of its caller or the event	almost always: a service can do no more than was approved, whoever calls it; granting a role is approved by that object's stewards
<code>caller</code>	whoever called it, with their own rights	a service people call to do what they could do themselves; never with triggers or schedules (they have no caller)
a user id (<code>erp</code>)	that integration user, with that user's roles	an outside system whose rights are managed as a person's

3.4 Governance: deny by default, approved by whoever answers

- **Callers.** Nobody calls a service until its **callers** name them (users, groups). A refused caller reads who may call and which stewards decide.

- **Reach.** A service touches only the objects and operations in `uses.objects` (`read`, `create`, `update`, `action`, `archive`), only the connections in `uses.connections`, and runs only the transactions in `uses.transactions`; it sends a connection only what the connection allows. Anything else is a fault: logged, refused.
- **Approval.** A service answers to its own stewards and to the stewards of what it reaches: every object it may write, reacts to, or holds a role on, and every connection it uses. An ERP confirmation of lot releases is approved by whoever stewards the connection and the lot.
- **Audit.** Every run, refused or not: who, how (web, UI, trigger, schedule), the input, the output, each request sent (connection, method, path, status, time; never a secret), and any refusal. Every record write inside it is audited as its identity, on behalf of the caller or the event.

4. A first integration, step by step

ERP sends work orders to OpenCore MES, and OpenCore MES confirms each released lot to ERP. Four designs in one change request: a connection, a web service, a trigger, their scripts. Then a token for ERP and the secret for the connection.

DV-01 Give ERP an identity and a token

Purpose	ERP calls as an integration user; its token says who is calling.
Done by	IT
Before you start	The user <code>erp</code> exists and is active (People & departments). The seed has one.

#	Do this	You should see	Done
1	On the server: <code>node app/mes/db/token.mjs erp "ERP production"</code> (with <code>DATABASE_URL</code> set to the plant's database).	A token <code>mes_...</code> , scope <code>service:call</code> , printed once. Only its hash is kept.	
2	Give the token to ERP's administrators through your secrets process.	—	

The token only says who calls. Which services that user may call is each service's design (its callers), approved like any change.

DV-02 Describe ERP as a connection

Purpose	The outside system, the only requests services may send it, and its credential by name.
Done by	Designer

#	Do this	You should see	Done
1	Designer → Services & connections → New connection , name <code>erp</code> , or Add to this change → + connection in an open change.	The connection's tabs: General, Authentication, Allowed requests, Activity, Stewards.	
2	General : label, base URL <code>https://erp.plant.local/api</code> ; timeout 3000 ms.	A URL with a user, a password, a query or a fragment is refused: credentials go in a secret.	
3	Authentication : <code>bearer</code> , secret <code>erp_token</code> .	Only the secret's name is in the design.	
4	Allowed requests : <code>POST /confirmations</code> .	Nothing else may be sent to ERP.	
5	Stewards : Engineering.	—	

```
{
  "name": "erp", "label": "ERP",
  "baseUrl": "https://erp.plant.local/api",
  "auth": { "kind": "bearer", "secret": "erp_token" },
  "allow": [{ "method": "POST", "path": "/confirmations" }],
  "timeoutMs": 3000,
  "stewards": ["engineering"]
}
```

DV-03 Write the web service ERP calls

Purpose	ERP sends a work order; it is created through the work order's own policy and rule pipe.
Done by	Designer
Before you start	The change from DV-02 is open.

#	Do this	You should see	Done
1	Add to this change → + service , name <code>erp_work_order_in</code> .	Its tabs: General, Input, Identity, Triggers, Reaches, Script, Try it, Activity, Stewards. A script is made with it.	
2	General : label, description; tick web service .	Its address will be <code>POST /svc/v1/erp_work_order_in</code> .	
3	Input : <code>wo_no</code> string, <code>item</code> string, <code>qty</code> decimal, <code>line</code> choice of L1, L2, L3, all required.	Each input is checked before the script runs; a bad one answers 400 with the field named.	
4	Identity : its own service role, holding <code>work_order planner</code> ; callers: the user <code>erp</code> .	It plans work orders, whoever calls it.	
5	Reaches : <code>work_order</code> : create.	—	
6	Script : as below.	Checked as you type: a syntax error on its line; a call it will not have (a typo, <code>fetch</code>) as a warning.	
7	Stewards : Production.	Will need approval from adds the work order's stewards.	

```
{
  "name": "erp_work_order_in", "label": "Work order from ERP",
  "description": "ERP sends a planned work order.",
  "input": {
    "wo_no": { "label": "Work order no.", "type": "string", "required": true },
    "item": { "type": "string", "required": true },
    "qty": { "type": "decimal", "required": true },
    "line": { "type": "enum", "values": ["L1", "L2", "L3"], "required": true }
  },
  "http": { "enabled": true },
  "callers": { "users": ["erp"], "groups": [] },
  "on": [],
  "runAs": "service",
  "roles": { "work_order": ["planner"] },
  "uses": { "connections": [], "objects": { "work_order": ["create"] } },
  "stewards": ["production"]
}
```

```
// ERP sends a work order; it is created as this service, through the work order's policy
and rule pipe.
export default async function erp_work_order_in(ctx) {
  const wo = await ctx.records.create("work_order", {
    wo_no: ctx.input.wo_no, item: ctx.input.item, qty: ctx.input.qty, line:
ctx.input.line,
  });
  ctx.output = { id: wo.id, wo_no: wo.wo_no, state: wo.state };
  return ctx;
}
```

DV-04 Write the trigger that confirms a released lot

Purpose	After a lot's release commits, ERP is told; retried while ERP is down.
Done by	Designer

#	Do this	You should see	Done
1	Add to this change → + service, name <code>erp_lot_released</code> ; not a web service.	—	
2	Triggers: Add trigger, object <code>lot</code> , event <code>transition:release</code> .	The events are <code>create</code> , <code>update</code> , <code>archive</code> , <code>restore</code> , or <code>transition:<action></code> of the object.	
3	Identity: the user <code>erp</code> . Reaches: <code>lot</code> : read; connection <code>erp</code> .	A trigger has no caller: it cannot run as its caller.	
4	Script: as below. A 5xx from ERP throws with <code>retry: true</code> ; any other refusal throws with words.	—	

```
{
  "name": "erp_lot_released", "label": "Lot released → ERP",
  "description": "Confirms a released lot to ERP.",
  "input": {}, "http": { "enabled": false },
  "callers": { "users": [], "groups": [] },
  "on": [{ "object": "lot", "event": "transition:release" }],
  "runAs": "erp",
  "uses": { "connections": ["erp"], "objects": { "lot": ["read"] } },
  "stewards": ["production"]
}
```

```
// A released lot is confirmed to ERP.
export default async function erp_lot_released(ctx) {
  const lot = await ctx.records.get("lot", ctx.event.id);
  const res = await ctx.http("erp", {
    method: "POST", path: "/confirmations",
    body: { lot_no: lot.lot_no, item: lot.item, qty: lot.qty, uom: lot.uom, event:
ctx.event.id },
  });
  if (res.status >= 500) throw Object.assign(new Error("ERP is unavailable"), { retry:
true });
  if (!res.ok) throw new Error(`ERP refused the confirmation (${res.status})`);
  ctx.output = { confirmation: res.body.id };
  return ctx;
}
```

DV-05 Dry-run the scripts and keep test cases

Purpose	Run the drafts as they will run, change nothing, and keep the runs as their evidence.		
Done by	Designer		
#	Do this	You should see	Done
1	Under <code>erp_lot_released</code> 's script, Dry run with an event { "kind": "transition:release", "object": "lot", "id": "<a lot id>" } and responses { "POST /confirmations": { "status": 201, "body": { "id": "DRY-1" } } }.	Its output { "confirmation": "DRY-1" }, the lot it read, and the request it would send, with the credential marked "not sent". Nothing reaches ERP.	
2	Again with { "POST /confirmations": { "status": 503 } }.	The error "ERP is unavailable", marked as asking for a retry.	
3	Name each run and press Save as test case .	The script's test cases list them.	
4	Under <code>erp_work_order_in</code> , dry-run with { "wo_no": "W0-2001", "item": "PA66", "qty": 0, "line": "L1" }.	Refused by the work order's own rule: "The quantity must be more than zero." A real create goes through the same pipe.	
5	Again with a quantity of 10; save both as test cases.	The work order it would create, and nothing created.	

A dry run runs as the service will: its own service role with the draft's roles, its user, or its caller (you, or one of its callers standing in). Acting with anyone's rights but your own is audited (`dry-run-as`).

DV-06 Submit, approve, and set the secret

Purpose	Nothing answers until the change is approved and executed; the outbound call needs its secret.
Done by	Designer; reviewer; department representatives; IT

#	Do this	You should see	Done
1	Before approval, call <code>POST /svc/v1/erp_work_order_in</code> .	404: nothing answers before it is published.	
2	Write the reason, run the fitness test, submit.	It passes: the scripts compile and call only what they will have; their test cases pass. Will need approval from: Engineering, Production and Quality: the stewards of the connection, of the services, and of the work order and the lot they reach.	
3	Review, then approve for each department.	Executed: the services are live at once, on every node.	
4	IT sets the secret on every node that may run the trigger: <code>MES_SECRET_ERP_TOKEN=<ERP's credential></code> , then restarts the node.	Secret names are lower case letters, digits and <code>_</code> , up to 48; the variable is the name in upper case after <code>MES_SECRET_</code> .	

DV-07 Call it from ERP, and watch it

Purpose	The inbound call, the outbound confirmation, and where to see both.
Done by	ERP's developer; designer

#	Do this	You should see	Done
1	<code>GET /svc/v1/openapi.json</code> with ERP's token.	An OpenAPI 3.1 description of exactly the services this user may call.	
2	Call it: <code>curl -X POST https://mes.plant.local/svc/v1/erp_work_order_in -H "Authorization: Bearer \$TOKEN" -H "Content-Type: application/json" -H "Idempotency-Key: order-2001" -d '{"wo_no":"W0-2001","item":"PA66","qty":10,"line":"L1"}'</code>	<code>200 { "output": { "id": "...", "wo_no": "W0-2001", "state": "planned" } }</code>	
3	Send the same request again, same <code>Idempotency-Key</code> .	The first answer again; no second work order.	
4	Release a lot (a person, a transaction, anything).	Within seconds ERP receives <code>POST /confirmations</code> , with the bearer credential.	

#	Do this	You should see	Done
5	Open Design → Integration monitor , and the services' Activity tabs.	The inbound calls, the outbound requests, the trigger's runs. A failed one can be Send again once ERP is back.	

5. Reference

5.1 Connection

Key	What	Rules
name	its name	lower case letters, digits and _
label	what people read	required
baseUrl	the system's address	http:// or https://; no user, password, query or fragment
auth.kind	how it authenticates	none, bearer, header, basic. bearer: Authorization: Bearer <secret>; basic: the secret is user:password, sent base64; header: the secret in the header auth.header (not host, cookie, content-length, transfer-encoding, connection)
auth.secret	the secret's name	lower case, digits, _; its value is MES_SECRET_<NAME> on the server
allow	the only requests services may send	each { "method", "path" }: GET, POST, PUT, PATCH, DELETE; a path starts with /, a trailing * allows what follows (/orders/*), nothing else is a pattern
timeoutMs	how long a request may take	100 to 30 000; 5 000 when left out
stewards	the departments that answer for it	at least one; they approve every service that starts using it

5.2 Service

Key	What	Rules
name	its name; its script has the same	lower case letters, digits and _
input	what a caller sends, typed like fields	types string, text, integer, decimal, boolean, date, enum, ref, image; required; an enum's values; a ref's to. Text is at most 2 000 characters (or the input's max); a date is YYYY-MM-DD; a ref is a record id
http.enabled	a web service	POST /svc/v1/<name>
callers	who may call it	{ "users": [], "groups": [] }: nobody until named
on	its triggers	{ "object", "event" } with create, update, archive, restore or transition:<action>; or a schedule (5.6)
runAs	who it acts as	service, caller or a user id (3.3); triggers and schedules need service or a user
roles	its service role	{ "<object>": ["<role>"] }, roles the object declares; it needs one on every object it reaches
uses.objects	what it may do to records	{ "<object>": [ops] }, ops of read, create, update, action, archive (archive is archiving and restoring)
uses.transactions	the transactions it may run	["<transaction>"]; run as its own service role, each transaction's callers.services names it too, and its stewards approve. Not a signed transaction

Key	What	Rules
<code>uses.connections</code>	the connections it may call	published connections, or ones in the same change
<code>runOn</code>	keeps its triggers and scheduled runs to nodes with this tag	lower case letters, digits, _ and -
<code>stewards</code>	the departments that answer for it	at least one

5.3 The script contract

One file, one function of the service's name, context in, context out. Nothing else in the file: no `import`, no other statement. The new script the designer makes:

```
// What this service does, in one line.
// ctx.input: what the caller sent · ctx.event: the record event, or { kind: "schedule",
// scheduledAt, previous } · ctx.user: who it acts as
// ctx.records.get/list/create/update/action/archive/restore(object, ...): as ctx.user, through
// policy and the rule pipe
// ctx.http(connection, { method, path, query, body }) → { status, ok, body }: only what the
// connection allows
// ctx.transactions.run(name, input, { key }) → { run, changes, records }: a transaction it
// may run (uses.transactions), all or nothing
export default async function my_service(ctx) {
  // throw Object.assign(new Error("Say what is wrong."), { field: "some_input" });
  ctx.output = { ok: true };
  return ctx;
}
```

ctx	What
<code>ctx.input</code>	what the caller sent, checked against the input; empty for a trigger or a schedule
<code>ctx.event</code>	a trigger's { kind, object, id, by } (kind is the event, "transition:release"); a schedule's { kind: "schedule", scheduledAt, previous: { at, output }, page }; null for a call
<code>ctx.user</code>	who it acts as: { id, name, onBehalfOf }
<code>ctx.now</code>	the time it started (ISO); use it, not the system clock
<code>ctx.service</code>	its name
<code>ctx.output</code>	what the caller gets back (a web service answers { output }); a scheduled run's is kept for the next (its cursor)
<code>ctx.records.get(object, id)</code>	a record, or null; needs <code>read</code>
<code>ctx.records.list(object, where)</code>	records whose fields equal <code>where</code> 's values; needs <code>read</code> . Found in the database among every record in use its identity may read, the 200 changed last of them; a field it may not read is never filtered on. Without <code>where</code> : the 200 changed last
<code>ctx.records.create(object, data)</code>	a new record; needs <code>create</code>

ctx	What
<code>ctx.records.update(object, id, data)</code>	changed fields; needs <code>update</code>
<code>ctx.records.action(object, id, action)</code>	a transition (<code>release</code>); needs <code>action</code>
<code>ctx.records.archive / restore(object, id)</code>	needs <code>archive</code>
<code>ctx.http(connection, { method, path, query, body })</code>	a request; answers <code>{ status, ok, body }</code> whatever the status (the script decides what a 404 means)
<code>ctx.transactions.run(name, input, { key })</code>	a transaction its design lists in <code>uses.transactions</code> , run as <code>ctx.user</code> : its callers (the service in <code>callers.services</code> , when it runs as its own role), its checks, its steps through each object's policies and rule pipe, all or nothing; answers <code>{ run, changes, records }</code> . A refusal throws with the transaction's words and fields. <code>key</code> (at most 60 characters: the lot's number, the event's id) makes a retried run answer the first. A transaction a person signs is never run by a service. In a dry run it is planned, not run

Every record call goes through the object's policies and rule pipe as `ctx.user`, and is audited. A refused write throws with the object's own words. About `ctx.http`:

- The path is relative to the connection's base URL, starts with `/`, has no `..`, `//`, `?`, `#` or backslash; parameters go in `query`.
- The body is sent as JSON (not with GET); the answer is parsed when it says it is JSON. Answers over 1 MB are refused.
- Redirects are never followed. The connection's credential is added on the server; the script never sees it.
- An unreachable system or a timeout throws a fault that asks for a retry.

5.4 Refusals, faults and retries

The script...	It is	A caller gets	A trigger's run is
throws <code>new Error("words")</code> , optionally with <code>field</code> or <code>fields</code>	a refusal	422 with the words and fields	rejected : retrying would say the same
has a record write refused (policy, rule pipe, state)	a refusal	422 (403, 404 or 409 as the records said)	rejected
throws with <code>{ retry: true }</code> , or a connection is unreachable	the other system is down	502	retry , with a growing wait, then dead
throws anything else (a <code>TypeError</code> , a call it may not make, a missing secret)	a fault: a mistake in the design or script	500 (the words go to the log, not the caller)	retry, then dead
returns its context	done	200 <code>{ output }</code>	done

A trigger's run is claimed for 2 minutes (a node that dies mid-run leaves it to another), and retried after 2, 4, 8, ... seconds (at most 5 minutes apart), up to 10 attempts; then it is **dead**, and the event log says so (`trigger.dead`). **Send again** on the service's **Activity** tab runs a retrying, dead or rejected one again.

5.5 Triggers: exactly once to enqueue, at least once to run

- A record write that matches a trigger writes its outbox row **in its own database transaction**: a committed release always sets the trigger off, a rolled-back one never.
- Every node's outbox worker claims due rows with `FOR UPDATE SKIP LOCKED`, ten at a time: several nodes share the work, none runs a row twice at once.
- The service runs as its **current** published version, as its identity, after the event.
- A service never sets itself off; a chain of services setting each other off stops at three.
- **Make the other side idempotent**. A run may, rarely, reach it twice (a node dies after ERP answered). Send the event's id (`ctx.event.id` with the event kind) as the other system's key.

5.6 Schedules

A schedule is a trigger the clock sets off: structured, not cron.

```
"on": [{
  "schedule": { "every": { "minutes": 15 }, "between": ["06:00", "22:00"],
    "days": ["mon", "tue", "wed", "thu", "fri"], "tz": "Europe/Berlin" },
  "missed": "last", "overlap": "skip"
}]
```

Key	Values
every	{ "minutes": 1–720 } or { "hours": 1–24 }, counted from midnight
at	times of day, ["06:00", "14:00"]
between	narrows every to a window; it may cross midnight
days	mon, tue, wed, thu, fri, sat, sun; all when left out
tz	an IANA time zone; the plant's (<code>PLANT_TZ</code>) when left out. A time daylight saving skips runs once when it would have been; one that happens twice runs once
missed	none, last, all (the default is last): runs that could not start within a minute of their time; all runs up to 100
overlap	skip, queue (the default is skip: nothing is queued while a run is waiting or running)
from	instead of every, at, between and days: "<suite>.<kind>", a kind of schedule an installed suite adds, whose times the suite works out (a shift calendar it keeps, say), with the settings that kind names beside it ({ "from": "<suite>.<kind>", "<setting>": value, "tz": "..." }). The designer lists the kinds installed. With the suite removed the service stays published, nothing is planned for that schedule, and the monitor says which suite it needs

A run is planned once, however many nodes plan; a schedule seen for the first time is planned from then on. `ctx.event.previous` is the last successful run ({ `at`, `output` }), so a script keeps its cursor in its output. A run that answers `output.more: true` gets its next page at once (`ctx.event.page`), up to 100 pages. Pause, resume and **Run now** are operations in the monitor: audited, not design changes.

A sync by the clock: its connection allows GET /orders , and it reaches work_order with read, create and update.

```
// Every 15 minutes: the orders ERP changed since the last successful run, a page at a time.
export default async function erp_orders_pull(ctx) {
  const since = ctx.event.previous?.output?.cursor ?? "2026-01-01T00:00:00Z";
  const res = await ctx.http("erp", { method: "GET", path: "/orders", query: { changedSince: since } });
  if (res.status >= 500) throw Object.assign(new Error("ERP is unavailable"), { retry: true });
  if (!res.ok) throw new Error(`ERP refused the order list (${res.status})`);
  for (const order of res.body.orders) {
    const [known] = await ctx.records.list("work_order", { wo_no: order.wo_no });
    if (known) await ctx.records.update("work_order", known.id, { qty: order.qty });
    else await ctx.records.create("work_order", { wo_no: order.wo_no, item: order.item, qty: order.qty, line: order.line });
  }
  ctx.output = { cursor: res.body.until, more: res.body.hasMore === true };
  return ctx;
}
```

5.7 The web service API

Call	POST /svc/v1/<service>, Content-Type: application/json, the input as the body (at most 256 KB)
Authenticate	Authorization: Bearer mes_...: a token with the scope service:call
Describe	GET /svc/v1/openapi.json: OpenAPI 3.1, only the web services this token's user may call; it changes as changes execute
Retry safely	Idempotency-Key: 8 to 100 characters. The same key from the same caller answers the first successful answer again, without running it
Answer	{ "output": ... }; refused: { "error", "fields"?, "code" }

Status	Means	code
200	done	—
400	bad input (each field named), or a body that is not JSON	service.input
401	no token, or a token that is not valid	token.missing
403	the token lacks service:call, or its user is not among the service's callers	scope.missing, service.denied
404	no such web service (not published, or not a web service)	—
413 · 415	the body is over 256 KB · not application/json	—
422	refused by the script, a rule or a policy: its words, and the fields	service.rejected or the rule's
500 · 502	the service failed · a system it depends on could not be reached	service.fault

5.8 Limits

What	Limit
A web service request body	256 KB

What	Limit
An answer from a connection	1 MB
A connection's timeout	100 ms to 30 s (default 5 s)
A service run	200 ms of CPU at a time, 15 s in all, 64 MB of memory; past them it is stopped and fails closed
A dry run	5 s
Trigger attempts	10, waiting 2, 4, 8, ... s, at most 5 minutes
A chain of services	3
Schedule catch-up	up to 100 missed runs, from the last 7 days
Pages of a scheduled run	100
<code>ctx.records.list</code>	with <code>where</code> , the 200 changed last of the matching records its identity may read, among all of them; without, the 200 changed last

5.9 Security

- **Scripts run apart.** Every rule and service script runs in the script runner: a process of its own beside each web process, started with Node's permission model (it reads only the app's files, writes none, starts no process) and an empty environment, so it holds no database address, secret or key. Jobs run in worker threads with a memory limit; one past its deadline is terminated. A script reaches records and systems only through `ctx`.
- **No network from a script** but `ctx.http`, through a connection's allow list. No `fetch`, no `Math.random`: the editor warns about both.
- **Secrets** are set on the server (`MES_SECRET_<NAME>`), never in a design, a review, the copilot's context or the audit. Changing a value is operations; changing which secret a connection uses is a design change.
- **Tokens** are shown once and kept as a hash; a designer revokes an AI token in **AI access**.
- **Public demos and sandboxes** reach no outside system: a call is refused, saying so.

The operating system's walls are each installation's to turn on. What the product does everywhere stops at Node's walls and the script's context. On Linux, add the kernel's: install bubblewrap (`apt-get install bubblewrap`) and, on Ubuntu 23.10 and later, its AppArmor profile (`install -m 0644 ops/script-runner-sandbox.apparmor /etc/apparmor.d/open-mes-bwrap && apparmor_parser -r /etc/apparmor.d/open-mes-bwrap`: those versions refuse user namespaces without one, which is also why `unshare -r -n` fails there); then set `SCRIPT_RUNNER_WRAP=ops/script-runner-sandbox.sh`. The runner then has a network namespace with loopback only, a read-only file system of the libraries, Node and the app's code, and no capabilities.

Check it by what the runner finds. At start it reports whether it sees any network and the sandbox it is in: the start log says `scripts: isolated (bwrap): no network, ...`, and `/healthz` shows `"scripts": {"network": "none", "sandbox": "bwrap", ...}`. `"network": "host"` means the walls are not there (a production instance also writes `scripts.unisolated` to its event log). Once they hold, set `SCRIPT_ISOLATION=required`: the instance refuses to start without them.

Caution. Until `SCRIPT_RUNNER_WRAP` is set, and `/healthz` shows `"network": "none"`, a script that found a way out of its context would have the host's network (though no credentials). Off Linux there is no wrapper.

5.10 Testing

- **As you type:** the script is parsed in the browser (never run there); syntax errors are marked on their line anywhere; calls the script will not have are warned about.
- **Dry run:** executed on the server as the service will run: real reads; writes through policy and the rule pipe, not saved; requests checked and answered from `responses` ({ "POST /confirmations": { status, body } }), never sent. A runtime error is marked on its line.
- **Test cases (Save as test case, or written):** { name, run: { input } | { event, responses }, as?, expect: { ok?, output?, error?, writes?, requests? } }. A new or changed script without one, or with a failing one, fails the fitness test, which runs them all on submit.
- **End to end:** `app/mes/test/integration.mjs` designs, approves and calls an ERP integration against a fake ERP (wrong caller, bad input, a rule's refusal, a retry by key, a confirmation while ERP is down); `app/mes/test/scheduler.mjs` proves the schedules. Both run in `npm run test:all`.

5.11 Running it: nodes, settings, the monitor

Setting	What
MES_SECRET_<NAME>	a connection's secret, on every node that runs its services
NODE_TAGS=erp,plant1	this node's tags; a service with <code>runOn: "erp"</code> has its triggers and scheduled runs run only on nodes tagged <code>erp</code> (inside the network that reaches ERP). Calls from outside run wherever they arrive
SCHEDULER=0	this node plans no schedules
OUTBOX=0	this node runs no triggers or scheduled runs
PLANT_TZ	the plant's time zone, for schedules that name none

Every node reports itself every 10 s. The **integration monitor** (`/design/integration`) shows the nodes (and warns when none plans schedules or runs the outbox, or none has a service's tag), the schedules (next and last run, totals, queue; Pause, Resume, Run now), inbound calls and outbound requests of the last 24 hours, and the record triggers' queues. The event log has `trigger.dead`, `schedule.missed`, `schedule.skipped`, `schedule.paused`, `schedule.resumed`, `schedule.suite_missing` (a schedule from a suite that is not installed) and `schedule.suite_failed` (an installed suite could not give a schedule's times: nothing was planned for it then).

6. Recipes

6.1 Inbound without duplicates

Have the caller send an `Idempotency-Key` per business message (the order number and its revision). Inside the script, guard on the natural key too, for a message sent again under another key:

```
const [known] = await ctx.records.list("work_order", { wo_no: ctx.input.wo_no });
if (known) { ctx.output = { id: known.id, state: known.state, existed: true }; return ctx; }
```

6.2 Saying no with the right field

Throw with the input it is about; a caller reads 422 with the field named, as a form does:

```
if (ctx.input.qty > 10000) throw Object.assign(new Error("At most 10 000 per order."), {
  field: "qty" });
```

6.3 Down is not no

Tell a system that is down (5xx, unreachable) from one that refuses (4xx). Throw with `retry: true` only for the first: a trigger is retried; a refusal is final, and its words are kept.

6.4 Choosing the identity

Prefer the service's own role: what it may do is on the service, approved by the stewards of each object it touches, and does not grow when someone gives the integration user another role. Use a named user when your plant manages the outside system's rights as a person's. Use the caller only for services people call to do, faster, what they may do anyway.

6.5 Keeping a run inside the right network

Give the nodes inside the network that reaches ERP a tag (`NODE_TAGS=erp`) and the service `runOn: "erp"`; the monitor warns when no live node has the tag.

7. Other ways in

7.1 The AI design API

An AI works in the designer as the person whose token it holds (§16.8): it reads, drafts, checks, tests and dry-runs, and never reviews or approves. Tokens are issued in **Designer → AI access**, with scopes of `design:read`, `design:draft`, `design:submit` (submitting is optional).

`GET /ai/v1/openapi.json` describes it; `node app/mes/test/ai-agent.mjs` is an agent's whole loop.

Route	Does
<code>GET /me · /contract · /catalog</code>	who the token is; the design contract (and the published contracts' names); the plant's model
<code>GET /contracts/{name}</code>	a published contract (<code>docs/contracts : equipment-adapter</code>), its specification, schema and conformance kit
<code>GET /objects/{name} · /scripts/{name} · /flows/{name}</code>	a live design
<code>GET · POST /changes; GET · PUT /changes/{id}</code>	list and start changes; read and save a draft
<code>GET /model · POST /model/preview · /model/changes</code>	every published design as one model file (no records, secrets or history); what a model file would change here; one change request from it
<code>POST /changes/{id}/include</code>	bring a live object, transaction, screen, flow, service or connection into a change, to change several together (one review, one approval per department)
<code>POST /validate · /scripts/{name}/test · /pipe/run · /access/simulate · /dry-run</code>	check a draft; run a script's tests; run a rule pipe; simulate access; dry-run a service
<code>POST /flows/check · /flows/layout · /flows/explain · /flows/walk · /scenarios/try</code>	flow templates and their scenarios; an input flow walked with sample values
<code>POST /changes/{id}/fitness · /changes/{id}/submit</code>	run the fitness test; submit (with the scope)

Requests are JSON; a token sending too many is answered 429.

7.2 Versions and notice

`/ai/v1` and `/svc/v1` keep what they promise for as long as `/v1` runs: the contract `docs/contracts/http-apis` writes it down (every `/ai/v1` operation with its scope and the fields it reads; the `/svc/v1` envelope, its statuses and its error body `{ error, code?, fields? }`). A new version may add; nothing promised leaves `/v1` (that is `/v2`, beside it).

- **Every answer** carries `API-Version: 1.0`.
- **Notice first.** What `/v1` will lose is marked deprecated at least 180 days before it goes; each answer to it then carries `Deprecation` (when it was deprecated), `Sunset` (the date after which it may go) and `Link` (its successor). Log them, and move before the sunset. Nothing is deprecated today.
- **A plant's web services** are the plant's promise to you. A change that would break a caller who called in the last 30 days is refused at the plant's review unless the plant gave notice first: the

service then answers you with the same headers until its sunset, and its OpenAPI description says until when and what to use instead.

Check an instance yourself, changing nothing: `node docs/contracts/http-apis/kit.mjs --url <the MES> --token <a token>` (add `--service <name> --input <json>` to try a call and its retry; `--json` for a program).

7.3 Excel import and export

Records go in and out as spreadsheets from **Data → Import / export**, through the same forms' checks, as the person importing: what an object's design allows (create, update, the key rows are matched by). See the user guide, section 13.

7.4 What is not an interface

The browser's own calls (`/api/...`) and the database are internals: they change without notice and are not to be built on. Integrate through web services, connections, the AI API, or files.

7.5 Planned

Planned. Versioned contracts (§31): `/svc/v1` and `/ai/v1` versioned with notice before anything they promise changes, an extension API for suites (`core-api`), and a conformance kit and machine-readable schema for each. **Equipment integration** (§30): adapters (SECS/GEM over HSMS first, then OPC UA and MQTT) turning machine events into records' states through the same rules and audit, with an adapter contract and a simulator. **Equipment data collection** (§15.1): high-rate machine data outside the MES, which receives only derived events. None of these is built.

8. Appendix

8.1 Review checklist for a service change

#	Check	OK
1	Its identity is the least that works: its own role with only the roles it needs, or a user whose roles are known.	
2	Its callers name exactly who may call it.	
3	uses holds only the objects, operations, connections and transactions the script uses.	
4	Each connection allows only the requests the services send.	
5	Down (retry) and no (words) are told apart; refusals name the field.	
6	The other side is idempotent for triggers: the event's id is sent.	
7	Its test cases cover a good run, a refusal, and the other system down.	
8	No secret, address or credential is in the script or the design.	
9	runOn keeps it to nodes that can reach its systems, and some live node has that tag.	

8.2 Where it is in the code

File	What
app/mes/server/integration.js	running services, the outbox worker, schedules, the /svc/v1 handler, the monitor
app/mes/client/definition.js	validateConnection, validateService, the footprint (who approves)
app/mes/client/schedule.js	the schedule format, its checks, next runs
app/mes/server/script-runner.mjs, script-worker.mjs	where scripts run, and their limits
app/mes/server/fitness.js	test cases and the fitness test
app/mes/server/ai-api.js	tokens, the /ai/v1 API
app/mes/db/token.mjs	issuing an integration user's token
app/mes/test/integration.mjs, scheduler.mjs	the end-to-end proof

8.3 Revision history

Revision	Date	What changed
A	2026-10-02	First issue: services and integration.
B	2026-10-03	Installing OpenCore MES: development, a server, upgrades, settings, scaling.
C	2026-10-03	A service's ctx.records.list(object, where) finds matching records among all of them, not only the 200 changed last.

Revision	Date	What changed
D	2026-10-05	OpenCore MES is no longer described as a proof of concept: the guide applies to the platform as built.